

FDE Xlerate

Week 1, Day 3

Benjamin Rezos

The problem

■ Task

- Port Day 2's hand-rolled agent into a framework
- LangGraph selected

■ Purpose

- Every framework (or no framework) has its strengths, weaknesses, and best application contexts.
- In a real client engagement, the client's existing architecture or preferences must decide the framework selection, not the FDE.

■ Business problem

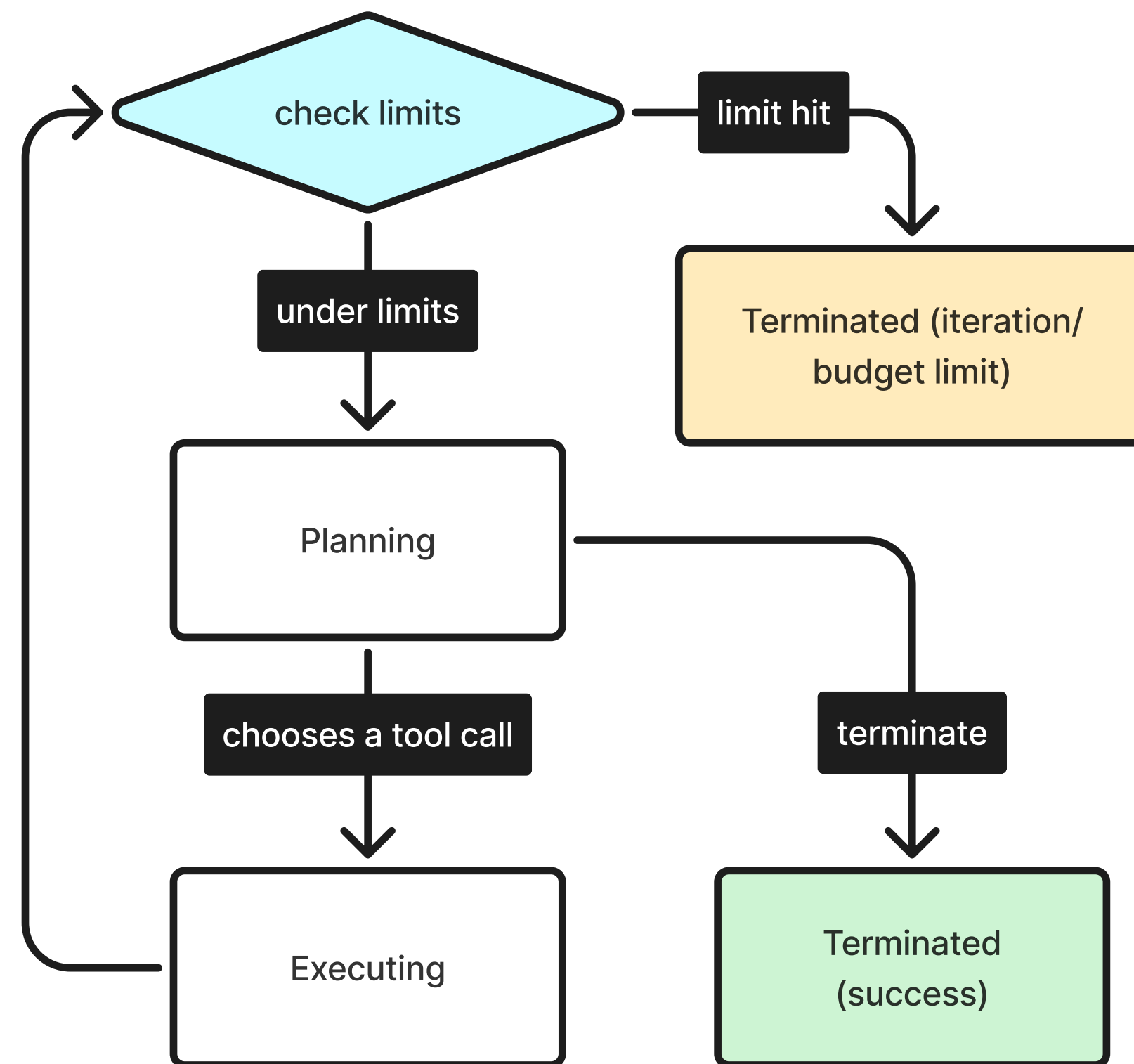
- Industry: insurance
- Case: Investigate a submitted insurance claim for fraud signals before it reaches a human adjuster.

When an insurance claim is made, it must be investigated for fraud signals to ensure legitimacy. Fraud signals may be spread across multiple data sources and any decision may rely on related information between sources. For example, claim history and max payout from a policy document may combine to indicate repeated instances of near-max claims.

A fraud investigation cannot be scripted as a workflow as decisions (conclusion made, more evidence needed, what evidence may support evidence-based hypothesis, etc.) must be made dynamically against each new piece of evidence pulled.

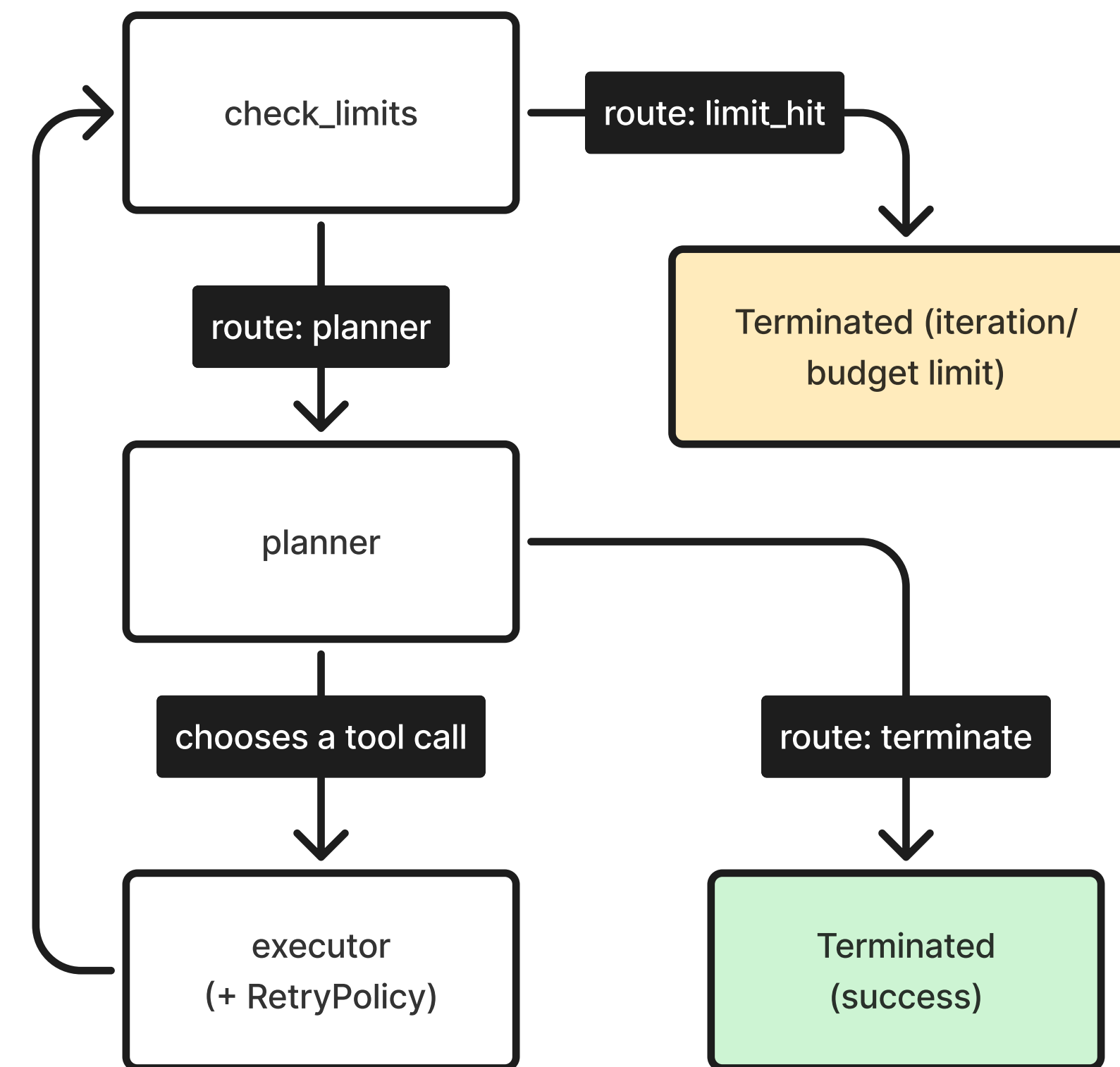
High-level flow

Day 2 (Hand-Rolled)



*Retry/ backoff handwritten inside
Executing*

Day 2 (LangGraph)



*Retry/ backoff = framework RetryPolicy
on the executor node*

Architecture - changed vs. reused

Reused unchanged

- planner.py - plan_next_action
 - Same system prompt, same model pin, same JSON contract
- tools.py
- registry.py
- mock_data.py
- errors.py

Replaced

- loop.py
 - executor.py - hand-rolled retry loop
- (This is where LangGraph's orchestration layer takes over)

Changed

- graph_state.py (the state schema)
- graph_nodes.py
 - Node functions + RetryPolicy
 - graph.py (the wiring)
- Two adapters were unavoidable, not optional:
 - a shim object for the planner's tokens_used mutation, and a node_attempt based check so an exhausted retry becomes an observation instead of crashing the graph

Tech Stack

■ Orchestration - LangGraph

low-level graph primitives (nodes + conditional edges), not the prebuilt ReAct-agent constructor, so the system prompt stays the same as day 2.

■ Retry

RetryPolicy (LangGraph's own retry primitive) attached to the executor node, configured to match day 2's hand-written curve (`initial_interval=0.5, backoff_factor=2.0, max_attempts=3, jitter=True`)

■ Tests

`tests/test_graph.py`, 6 fake-planner tests ported from day 2's `test_loop.py`, using a fast RetryPolicy variant since the framework's retry sleep can't be swapped out the way day 2's injectable sleep parameter could

■ Model

Claude Sonnet 5 via the Anthropic API as the planner (pinned to the exact same `0.104.1` day 2 uses)

■ Environment

An isolated venv (`requirements.txt`) rather than day 1/2's global install. langgraph never touches the global Python, and the setup is what let this actually be verified from a clean, from-scratch install rather than just "works in the venv already sitting here."